University of Victoria

ECE 455 – Real Time Control Systems

Project 1: Traffic Light RTOS

Date	March 2, 2026
Group Members	Nicole Chan → V01009561
	Evy Wind-Granot → V01013467

Project 1: Traffic Light RTOS **1**
1. Introduction 3
Figure 1. The traffic light system User Interface 3
Cars 3
Potentiometer 3
Traffic Light 3
2. Design solution 4
2.1. System Overview Diagram 4
Figure 2. System Overview Diagram 4
2.2. Middleware and Hardware Configs 5
Table 1. Pins Available on the Server 5
2.2.2. GPIO Configuration 6
Figure 3. gpio_init() codesnippet entailing the hardware initialization 6
2.2.3. Potentiometer Configuration 6
Figure 4. read_pot_adc() codesnippet demonstrating the behaviour 7
2.2.4. Shift Register 7
Figure 5. 74HC164-style shift registers layout 7
Figure 6. screen_write() function showing the full logic of the shift register displaying "data" 8
2.3. Application Software Design 8
2.3.1. Task 1: flow_adjust_task 8
2.3.2. Task 2: light_controller_task 8
Figure 7. Light duration calculation 9
2.3.3. Task 3: traffic_gen_task 9
Figure 8. Spawn Countdown codesnippet 9
Figure 9. A flow chart of the traffic generating algorithm 10
2.3.4. Task 4: display_Task 11
2.4. Initial Design Document & Comparison to Final Design 12
Figure 11. Initial design solution for project, including both hardware and software components 12
Figure 12. Final design of the hardware 13
3. Discussion 14
4. Limitations and Possible Improvements 14
5. Summary 15
6. Appendix 16

1. Introduction

Project 1 of ECE 455, Real Time Computer Systems revolves around creating the hardware/ electrical, and software components of a traffic light system that simulates cars going through a uni-directional, single lane traffic path, in a real time environment.

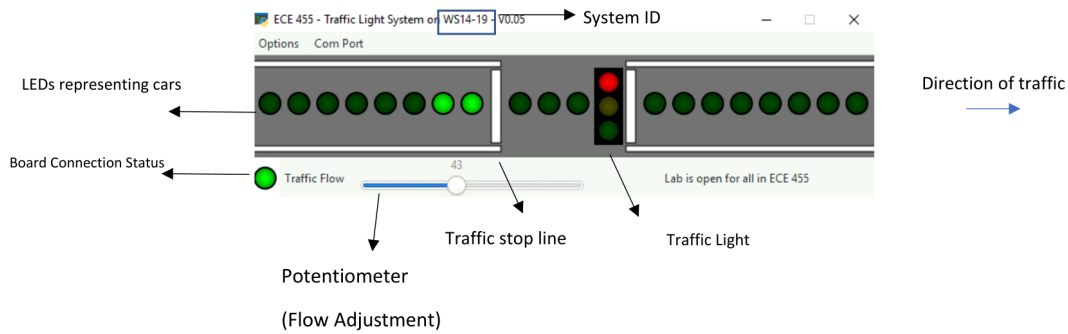

Figure 1. The traffic light system User Interface.

The general idea is as follows: there are 19 car positions that are "controlled" by a central traffic light, and whos' flow is controlled by a potentiometer reading. A light being on in a position signifies that position is actively occupied by a car. The 19 positions are split into 3 sections: 1-8, 9-11 and 12-19. Each section has its own behaviour that will be expanded in the remainder of the report. This traffic light system utilizes 3 primary components:

Cars

As mentioned above, there are 19 LEDs that each represent a position on the road that a car may occupy. If the light is on, then that position on the road is filled. This formation of the cars gives the illusion that the cars/LEDs are moving along the road, as the cars are shifted/moved incrementally, one by one, along the road.

Potentiometer

The potentiometer provides the ability to adjust manually the flow of traffic, or frequency of spawning new cars, and doing so while running the program and changing the flow dynamically at run-time.

Traffic Light

The traffic light system consists of 3 LEDs, red, yellow and green, which were used to govern the traffic at the intersection. Out of the 19 positions that the cars could be in; if they were in the first 8 positions and as long as the position in front of it was not occupied by another car (light was turned off) they were free to travel forward. The cars continue into the 9th position if and only if the traffic light is green, otherwise it stops and waits in the 8th position until the light turns green, meanwhile cars would line up behind them. If a car is in the 9-11 positions when the light turns

yellow, then they may continue into the last 8. In these last 8 no matter the state of the traffic light they may complete the circuit.

2. Design solution

2.1. System Overview Diagram

The system overview diagram, figure 2 below, demonstrates the interactions and relations of the tasks and queues present in the system. The system begins by getting the potentiometer ADC value to feed the flow_adjust_Task, which dictates the flow of the system. Furthermore, the light_controller_Task receives timer-expiry events from the one-shot software timer (light_timer) feed to it via the light_event_queue. After this we have the traffic_gen_Task and the light_controller_Task pushing their states to the display_task, which writes to the hardware, it is the only task that writes to hardware.

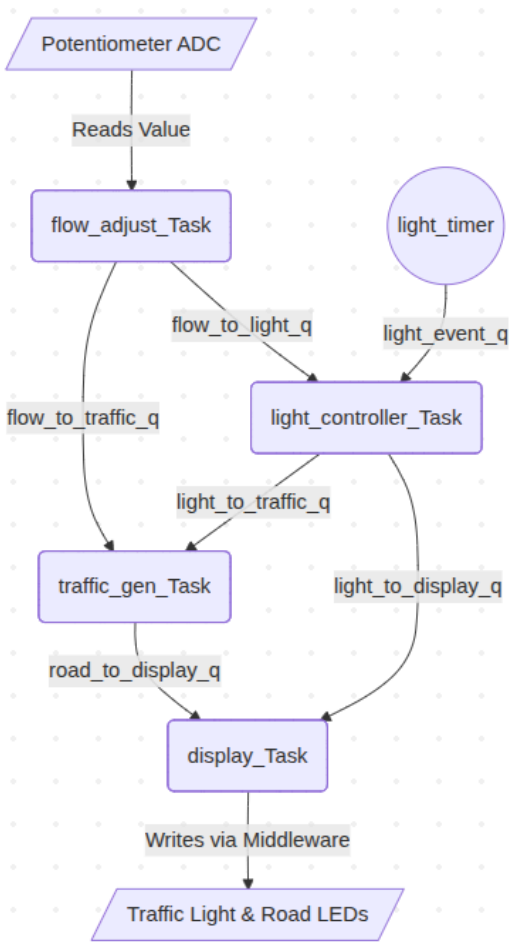

Figure 2. System Overview Diagram

As shown in figure 2. there are six queues for the communication between the four tasks, below is a summary of these six queues:

1. Flow_to_light_queue → Carries the 16-bit potentiometer ADC value from flow_adjust_task to light_controller_task
2. Flow_to_traffic_queue → Carries the 16-bit potentiometer ADC value from flow_adjust_task to traffic_gen_task
3. light_event _queue → Carries a single byte event token from the light_timer ISR to light_controller_task which signals that the light phase has changed.
4. Light_to_traffic_queue →Carries 8 bits of light states of the green, yellow and red lights to help with the stop-line rule.
5. Light_to_display_queue → Carries the light state from the light_controller_task to the display_task
6. Road_to_display_queue → Carries the 32-bit car bitmask from traffic_gen_task to display_task.

2.2. Middleware and Hardware Configs

The middleware had the purpose of configuring the STM32F4's GPIOC peripherals and ADC, using the provided libraries. The gpio_init() function is in charge of doing all the peripheral setup, and is called in the main() even before the scheduler starts.

2.2.1. GPIO Pins Used

There were seven pins utilized for the traffic light system, all mapped to Port C (GPIOC). Below is a table summarizing the pins used, signal, and notes on the pins functionality.

Table 1. Pins Available on the Server

STM32F0 Port Pin	Signal	Notes
PC0	Red Light	Traffic Light to signify to the cars to stop
PC1	Amber Light	Traffic Light to allow the cars in the intersection to pass through and for the others to prepare to stop
PC2	Green Light	Traffic Light to signify to the cars that they are free to go
PC3	Poteniometer/ADC	0-3 Volt Input, to control the flow of cars travelling through the system.
PC6	Shift Register Data	After clock falling edge data hold time of a minimum 1 micro second.
PC7	Shift Register Clock	Triggers the Falling edge
PC8	Shift Register Reset	Active low, minimum 1 mircosecond period.

2.2.2. GPIO Configuration

This behaviour is clearly defined in the gpio_init() from the code in the figure below.

```
// Initialize hardware
static void gpio_init()
{
    RCC_AHB1PeriphClockCmd(RCC_AHB1Periph_GPIOC, ENABLE);   //enable clock for portC
    RCC_APB2PeriphClockCmd(RCC_APB2Periph_ADC1, ENABLE);    //enable clock for ADC1

    //LED's
    GPIO_InitTypeDef GPIO_Output_Config;
    GPIO_StructInit(&GPIO_Output_Config);                   //reset all members of the struct to d
    GPIO_Output_Config.GPIO_Pin = (YEL | GRN | RED | CLR_PIN | CLK_PIN | DATA_PIN);     //we w
    GPIO_Output_Config.GPIO_Mode = GPIO_Mode_OUT;           //we want to control the LEDs, so we n
    GPIO_Output_Config.GPIO_PuPd = GPIO_PuPd_NOPULL;        //no pull-up or pull-down resistors
    GPIO_Output_Config.GPIO_Speed = GPIO_Speed_2MHz;        //the speed of the output signal is no
    GPIO_Output_Config.GPIO_OType = GPIO_OType_PP;          //push/pull
    GPIO_Init(GPIOC, &GPIO_Output_Config);

    //ADC
    GPIO_InitTypeDef GPIO_ADC_Config;
    GPIO_StructInit(&GPIO_ADC_Config);                  //reset all members of the struct to defau
    GPIO_ADC_Config.GPIO_Pin = GPIO_Pin_3;              //we want to read the value of the potenti
    GPIO_ADC_Config.GPIO_Mode = GPIO_Mode_AN;           //we want to read the value of the potenti
    GPIO_ADC_Config.GPIO_PuPd = GPIO_PuPd_NOPULL;       //no pull-up or pull-down resistors
    GPIO_ADC_Config.GPIO_Speed = GPIO_Speed_2MHz;       //the speed of the output signal is not im
    GPIO_Init(GPIOC, &GPIO_ADC_Config);
    ADC_CommonInitTypeDef ADC_Common;           //we need to configure the common ADC settings, su
    ADC_CommonStructInit(&ADC_Common);          //reset all members of the struct to default value
    ADC_Common.ADC_Mode = ADC_Mode_Independent; //use ADC1 only
    ADC_Common.ADC_Prescaler = ADC_Prescaler_Div4; //reduce ADC clock
    ADC_Common.ADC_DMAAccessMode = ADC_DMAAccessMode_Disabled; //no DMA
    ADC_Common.ADC_TwoSamplingDelay = ADC_TwoSamplingDelay_5Cycles; //default delay
    ADC_CommonInit(&ADC_Common);


    ADC_InitTypeDef ADC_Config;
    ADC_StructInit(&ADC_Config);                        //reset all members of the struct to d
    ADC_Config.ADC_ContinuousConvMode = DISABLE; //single conversions on demand
    ADC_Config.ADC_ExternalTrigConvEdge = ADC_ExternalTrigConvEdge_None; //software trigger on
    ADC_Config.ADC_DataAlign = ADC_DataAlign_Right;         //we want the data to be right-ali
    ADC_Init(ADC1,&ADC_Config);
    ADC_Cmd(ADC1, ENABLE);                      //enable the ADC
    ADC_RegularChannelConfig(ADC1, ADC_Channel_13, 1, ADC_SampleTime_3Cycles);  //we want to r
}
```

Figure 3. gpio_init() codesnippet entailing the hardware initialization

2.2.3. Potentiometer Configuration

The ADC was configured to be PC3. The function read_pot_adc() starts a single ADC conversion and returns the 12-bit value between 0-4095. The function is provided in the figure below.

```c
// Reads the ADC
static uint16_t read_pot_adc(void)
{
    /* Start a single ADC conversion and return the 12-bit value. */
    ADC_SoftwareStartConv(ADC1);
    while (ADC_GetFlagStatus(ADC1, ADC_FLAG_EOC) == RESET) {    //wait for conversion to complete
    }
    return (uint16_t)ADC_GetConversionValue(ADC1);
}
```

Figure 4. read_pot_adc() codesnippet demonstrating the behaviour

2.2.4. Shift Register

The traffic light system utilizes 3 x 74HC164-style shift registers, for which their layout is in figure 5 below. They act as 19-output serial-to parallel converters for the cars. To assist with this, the screen_write() function clocks out the 19 car bits MSB-first. In this function, the CLK pin is reset, and the DATA is either high or low, then the CLK is pulsed. data_Rst() causes the CLR pin low for just 10ms to clear all the LEDs, preparing for a new frame to be written in. This function is found in the figure below. The data, PC6, was connected to the A of the first shift register. The A and B were linked directly to eachother. Q_A-Q_F were connected to the LEDs, and the Q_H was connected to the A of the next shift register. The GND was connected to the ground rail, and V_{CC} was connected to the power rail. PC7, CLK was daisychained between all three shift registers. Finally, PC8, CLR' was also chained between all three shift registers.

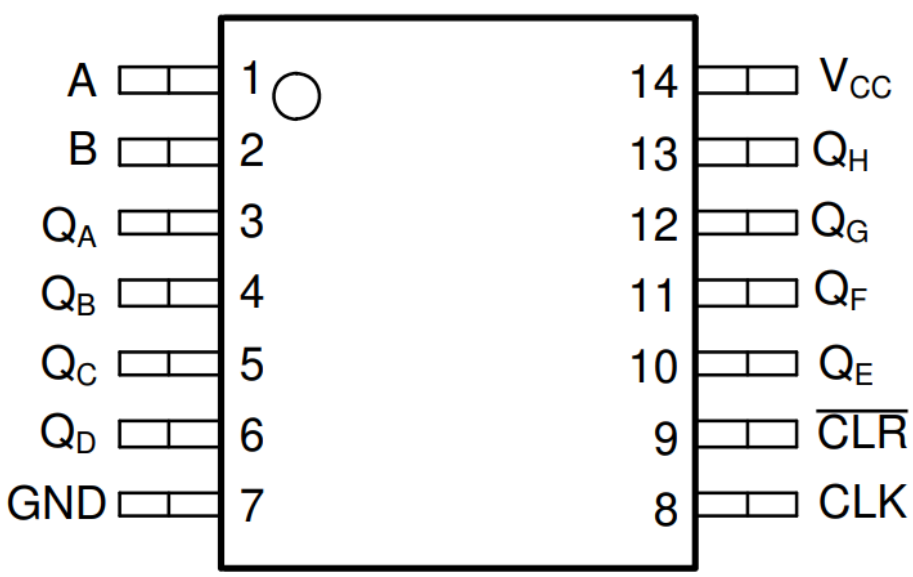

Figure 5. 74HC164-style shift registers layout

```c
/*  Writes car data to the shift register.
    The data is a 19-bit value, where each bit represents a car in the 3 lanes.
    The MSB represents the oldest car, and the LSB represents the newest car.
    A bit value of 1 means there is a car, and a bit value of 0 means there is no car */
void screen_Write(uint32_t data){
    //data_Rst(); // clear display
    char car_To_Send = 0x0;

    for(int i=18;i>=0;i--){ //MSB first, so we start from the highest bit and shift down to the lowest bit
        car_To_Send = 0x1 & (data>>i);
        if(car_To_Send == 0x1){
            data_High();
        }else{
            data_Low();
        }
    }
}
```

Figure 6. screen_write() function showing the full logic of the shift register displaying ***"data"***

2.3. Application Software Design

This section will break down the four primary tasks involved for this project:

2.3.1. Task 1: flow_adjust_task

The flow_adjust_task, is responsible for bridging between reading the pot value (every 100ms) using read_pot_adc() to sample the potentiometer and feeding that ADC value to flow_to_traffic_queue and flow_to_light_queue. This is done using the xQueueOverwrite() function that intentionally drops any unread value, replacing it instantly with the latest reading. This is critical to ensure real-time behaviour, and removing stale states. Another change that was learned along the way was assigning this task a higher priority, 2, than the generator and display tasks which both have priority 1. This ensures that the potentiometer readings are read even in high load situations.

2.3.2. Task 2: light_controller_task

This task is responsible for traffic light phase movement and it changes the traffic light state. Every process is initialized with a RED state, and this state is sent to the light_to_display_queue and light_to_traffic_queue, then starts a one-shot timer with a max red duration of 10,000ms.

Each iteration of the loop begins by waiting for a timer event to change the light state which doesn't block waiting for flow updates. Once this time elapses one bit changes and the flow reading assists in dictating the green and red light duration in ms.

```c
for (;;) {
    uint16_t flow_update = 0;
    if (flow_to_light_q != NULL && xQueueReceive(flow_to_light_q, &flow_update, 0) == pdPASS) {
        flow_value = flow_update;
    }
    // wait for a timer event to change the light state, but don't block waiting for flow updates
    if (light_event_q != NULL && xQueueReceive(light_event_q, &timer_event, portMAX_DELAY) == pdPASS) {
        /* Map flow to green/red durations. */
        uint32_t green_ms = LIGHT_GREEN_MIN_MS +
                            ((uint32_t)flow_value * (LIGHT_GREEN_MAX_MS - LIGHT_GREEN_MIN_MS)) / FLOW_MAX_VALUE;
        uint32_t red_ms = LIGHT_RED_MAX_MS -
                            ((uint32_t)flow_value * (LIGHT_RED_MAX_MS - LIGHT_RED_MIN_MS)) / FLOW_MAX_VALUE;
```

Figure 7. Light duration calculation.

The linear mapping of the ADC reading directly to phase duration. With the green light being double that of the red (10000ms vs 5000ms), this ensures a 2:1 green to red ratio no matter the flow value. The yellow light is held standard at 1500ms. Once the durations are found, the task continues to move through the states, restarts the timer for the new duration and broadcasts the new state using the xQueueOverwrite() function to light_to_display_queue and light_to_traffic_queue.

2.3.3. Task 3: traffic_gen_task

The traffic_gen_task runs at car_step_ms (800ms) using vTaskDelayUntil(). At each iteration flow_to_traffic_queue and light_to_traffic_queue are polled, then the road_step() is called moving all the cars one position forward, and conditionally, a new car is generated at the start of the road. Then the new updated position mapping is written to road_to_display_queue.

The road logic is handled in this function as well. When the light is green all 19 positions are simply shifting by 1. When yellow or red, the road is split at the stop_line_index, which is at 7(the stop line).

- Positions 0-7 can advance freely as long as the next position is available
- Positions 8-18 advance unconditionally since they have passed the stop line.

The car spawning mechanism is an XOR-shift PRNG seeded with a basic 0x1234ABCD. A random value is masked with the FLOW_MAX_VALUE, and a car spawns if it is less than flow_value, this enforces the proportionality of the spawning to the ADC value. The specific code that demonstrates this is below:

```c
    /* Guard: if flow exceeds max, clamp it before subtraction */
    uint16_t clamped_flow = (flow_value > FLOW_MAX_VALUE) ? FLOW_MAX_VALUE : flow_value;
    spawn_countdown = (uint8_t)(1U + ((FLOW_MAX_VALUE - clamped_flow) * 5U) / FLOW_MAX_VALUE);
    if (spawn_countdown == 0U) {
        spawn_countdown = 1U;
    }
}
```

Figure 8. Spawn Countdown codesnippet.

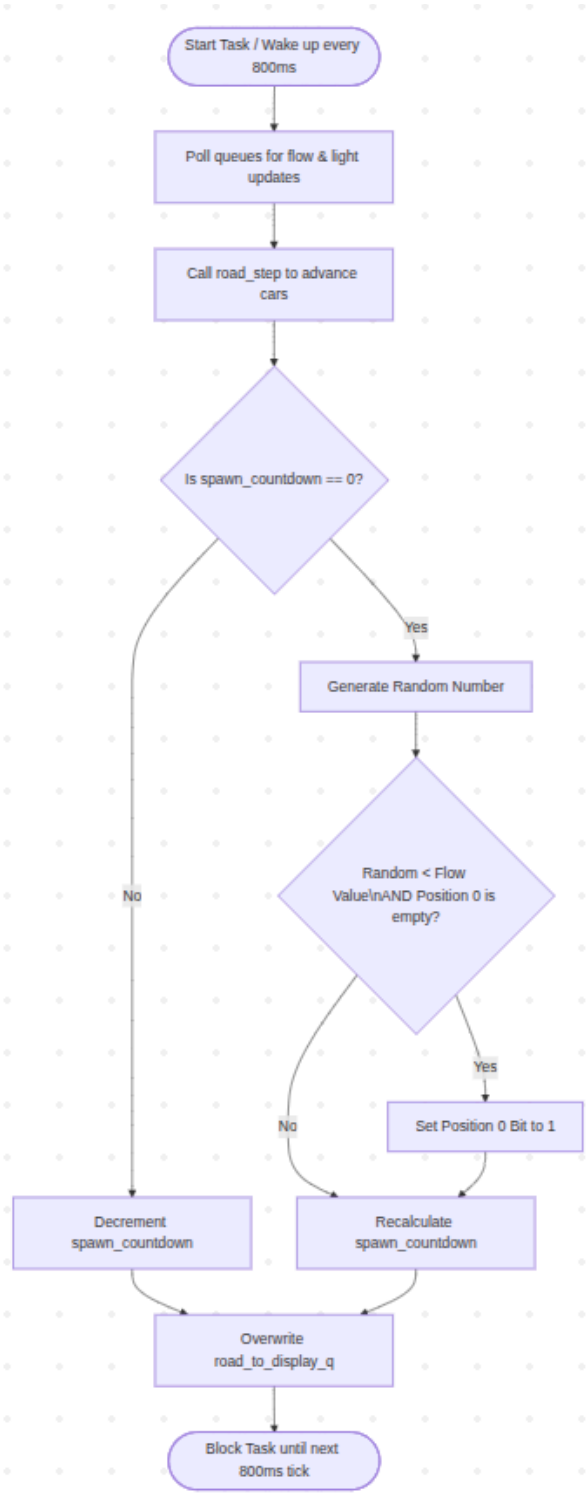

Figure 9. A flow chart of the traffic generating algorithm.

2.3.4. Task 4: display_Task

The display task runs at display_refresh_ms (800ms) using vTaskDelayUntil(), synchronised with traffic_gen_task. Each iteration polls the road_to_display_queue and light_to_display_q, retaining the previous value if no update has arrived. The task will then reset the traffic-light GPIO pins, then set the right traffic light to High, lastly it calls screen_write() which shows the car pattern using the shift registers. This task was specially designed to be the only task that touches the hardware outputs, to provide clear separation between software and hardware.

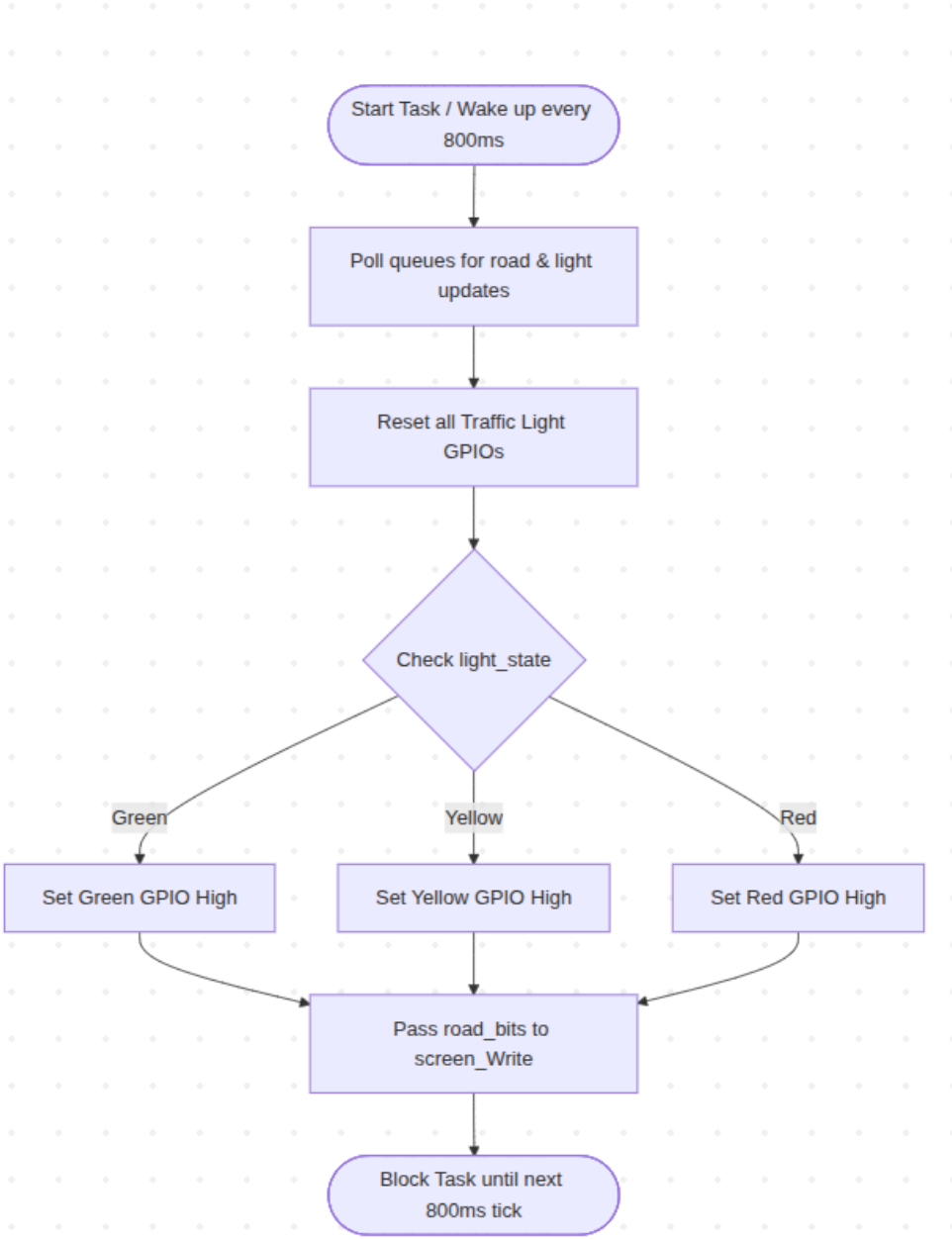

Figure 10. A flow chart of the system display algorithm.

2.4. Initial Design Document & Comparison to Final Design

Before starting any wiring and code, the figure below was produced. This document was meant to guide our development, direct us, and help us fall back on something if we got lost. The plan was to have 4 tasks, 3 queues and it also entailed the wiring/pin assignment. Overall the structure held quite closely to the final product.

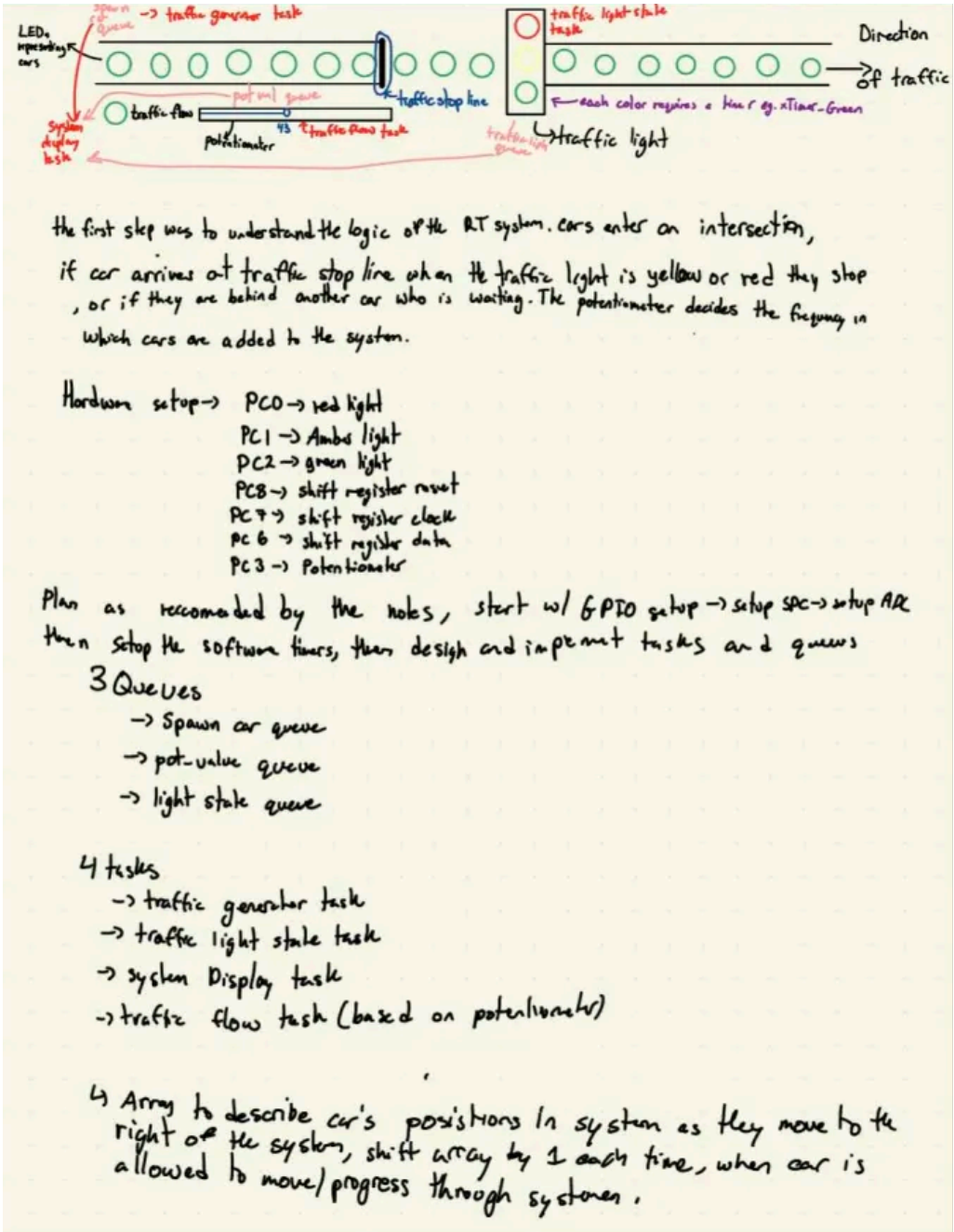

Figure 11. Initial design solution for project, including both hardware and software components

Some two major differences between the initially planned design and final product are:

- **Queue count:** In the initial design it was suspected that only 3 queues were to be used, one for spawning cars, one for pot value and one for the light state. In the end 6 queues were utilized. This design was chosen to allow for more precise and ease for the flow of information between tasks, limiting the ambiguity of a single pot value queue accessed by various sources.
- **Cars**: In the initial design above, the cars were presumed to be represented in an array. The final implementation amended this to use a 32-bit integer bitmask. This makes the shifting of the LEDs a single bitwise operation rather than an entire array coping process, making it much more efficient.

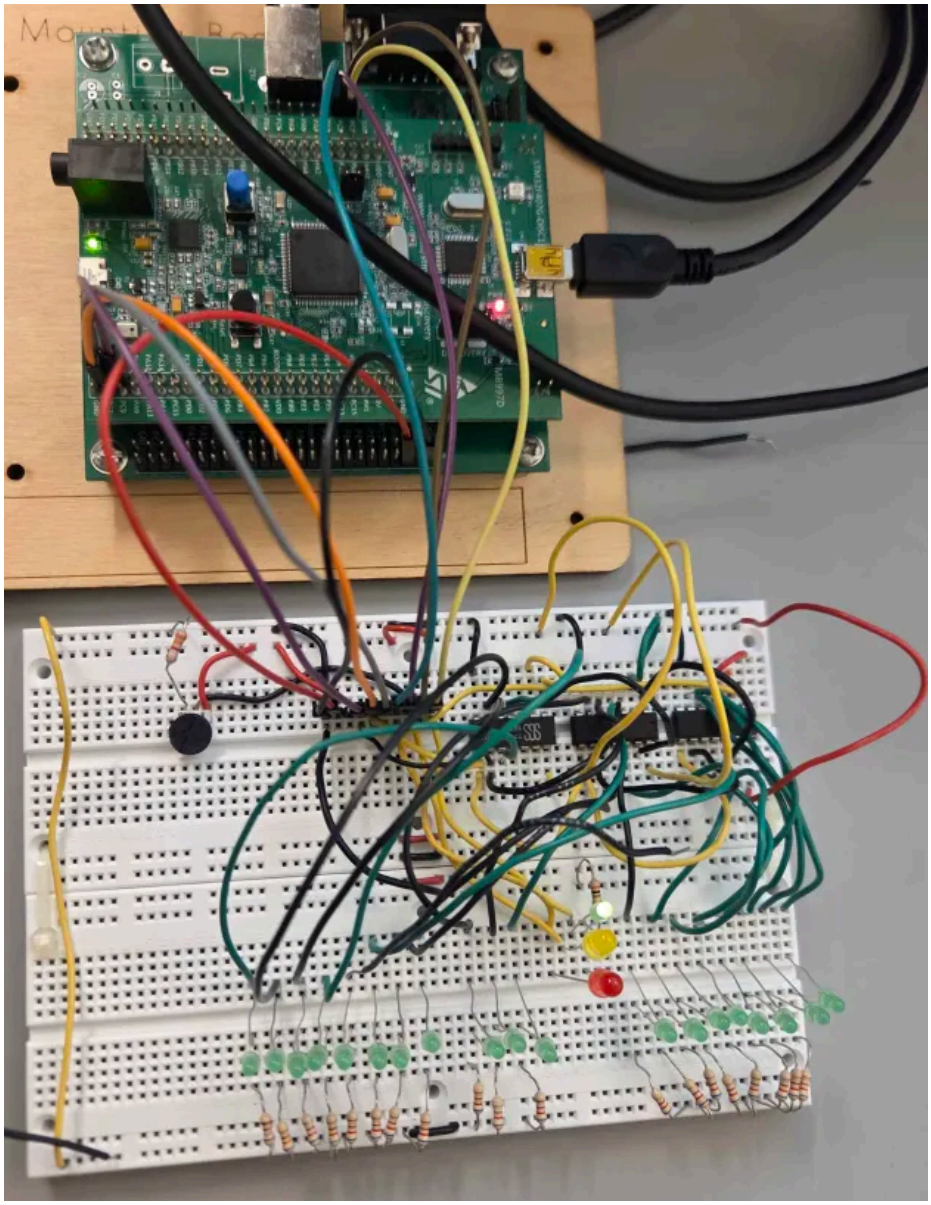

Figure 12. Final design of the hardware

3. Discussion

The Traffic Light System met the main functional and technical requirements detailed in the project specification. The system was built using four FreeRTOS tasks that only communicate through queues and software timers controlling the traffic light state transitions.

The Traffic Flow Adjustment Task continuously read the potentiometer using the ADC and sent the updated value to both the Traffic Generator and Traffic Light State tasks. Dedicated queues using xQueueOverwrite were used so each task always has the most recent reading without older values building up in the queue.

The Traffic Light State Task controlled the green, yellow, and red transitions using a software timer. The lengths of the green and red lights were dynamically adjusted depending on the traffic flow rate. When traffic flow increased, the green light duration became longer while the red light shortened, and at lower flow rates, the opposite occurred. The yellow light duration remained constant to meet the proportional timing requirements.

The vehicle movement and intersection behaviour were handled by the road_step function in the Traffic Generator Task. During a green light, each car moved forward one position on every tick and when the light turned yellow or red, the cars stopped at the intersection while vehicles that were already past the intersection kept moving forward. The shift registers refreshed every 800ms, updating the road state. It was then sent to the Display Task through a queue and updated with the appropriate traffic light LED, which allowed cars to smoothly move across the 19 led lane.

4. Limitations and Possible Improvements

A few limitations became noticeable during testing. Since traffic generation is random, lower flow rates do not always produce perfectly spaced vehicles. Although the system generally maintains the expected gaps between cars, there is no strict guarantee of consistent spacing. This could be improved by implementing a more controlled generation method that tracks the position of the last car and enforces a minimum distance before adding another one.

Another limitation is that the display refresh rate and vehicle movement speed depend on the delay used in the System Display Task. If the ADC sampling rate rate is slower than the display update rate, there can be a brief delay where the traffic light changes before the display updates. Possible improvements would be increasing the priority of the Traffic Light State Task or using a semaphore to better synchronize display updates with state refreshes.

On the hardware side, the circuit also became difficult to work with as more components were added. With 22 LEDs, three shift registers, and a potentiometer, the breadboard accumulated

many wires which made wire tracing and troubleshooting difficult. This could be improved by using consistent colour-coding of the wires and better wire organisation.

An additional improvement to the system would be to expand it to support multiple lanes or traffic that would go both ways (bidirectional). This would require adding another Traffic Generator instance and updating the Display Task so it can handle multiple lane states.

5. Summary

For this project, we created the hardware and software components of a traffic light system that simulates cars going through a single lane traffic path using a STM32F4 Discovery board and used FreeRTOS to manage the system's tasks. To drive the 19 car LEDs, we daisy-chained three 74HC164 shift registers together and used GPIO pins to control the traffic lights. We also used the ADC to read values from the potentiometer.

The system used four FreeRTOS tasks: Traffic Flow Adjustment, Traffic Generator, Traffic Light State, and System Display which communicated entirely through queues, meeting the requirement to avoid using global-variables. Software timers were used to control the light phases, with green and red durations changing based on the traffic flow rate.

All project requirements were achieved. Traffic was generated randomly based on the potentiometer input, vehicles moved between 1 and 3 LEDs per second, cars stopped correctly at the intersection during yellow or red lights, and traffic light timing followed the required green-to-red ratios at both maximum and minimum flow rates.

6. Appendix

```c
/* Standard includes. */
#include <stdint.h>
#include <stdio.h>
#include <stdlib.h>
#include "stm32f4_discovery.h"
/* Kernel includes. */
#include "stm32f4xx.h"
#include "../FreeRTOS_Source/include/FreeRTOS.h"
#include "../FreeRTOS_Source/include/queue.h"
#include "../FreeRTOS_Source/include/semphr.h"
#include "../FreeRTOS_Source/include/task.h"
#include "../FreeRTOS_Source/include/timers.h"

//GPIO & ADC includes
#include "../Libraries/STM32F4xx_StdPeriph_Driver/inc/stm32f4xx_gpio.h"
#include "../Libraries/STM32F4xx_StdPeriph_Driver/inc/stm32f4xx_rcc.h"
#include "../Libraries/STM32F4xx_StdPeriph_Driver/inc/stm32f4xx_adc.h"


/*-----------------------------------------------------------*/
#define mainQUEUE_LENGTH 100

// GPIO pin definitions
#define YEL GPIO_Pin_1
#define GRN GPIO_Pin_2
#define RED GPIO_Pin_0
#define ADC_PIN GPIO_Pin_3
#define LED_YEL GPIOC,GPIO_Pin_1
#define LED_GRN GPIOC,GPIO_Pin_2
#define LED_RED GPIOC,GPIO_Pin_0
#define POT GPIOC,GPIO_Pin_3

// shift register pin definitions
#define CLR_PIN GPIO_Pin_8  //reset
#define CLK_PIN GPIO_Pin_7  //clock
#define DATA_PIN GPIO_Pin_6 //data
#define SHIFT_PORT GPIOC

/*
 * Hardware specific clock configuration
 * that was not already performed before main() was called.
 */
static void gpio_init(void);

/*
 * The queue send and receive tasks as described in the comments at the top of
 * this file.
 */
static void flow_adjust_Task( void *pvParameters );
static void traffic_gen_Task( void *pvParameters );
static void light_controller_Task( void *pvParameters );
static void display_Task( void *pvParameters );
```

```c
/* Helper Functions */
static uint16_t read_pot_adc(void);
static void light_timer_callback(TimerHandle_t timer);
static uint32_t road_step(uint32_t road_bits, uint8_t light_state);
static uint32_t rng_next(uint32_t *state);
static void data_High(void);
static void data_Low(void);
static void data_Rst(void);
static void screen_Write(uint32_t data);
void delay(uint32_t ms);
typedef enum {  //Light state typedef
    LIGHT_GRN = 0,
    LIGHT_YEL = 1,
    LIGHT_RED = 2
} light_state_t;

/* Queues used for inter-task communication (no globals for sharing state). */
static QueueHandle_t flow_to_traffic_q = NULL;
static QueueHandle_t flow_to_light_q = NULL;
static QueueHandle_t light_to_traffic_q = NULL;
static QueueHandle_t light_to_display_q = NULL;
static QueueHandle_t road_to_display_q = NULL;
static QueueHandle_t light_event_q = NULL;
static TimerHandle_t light_timer = NULL;

/* Tunable system parameters. */
#define FLOW_MAX_VALUE 2400U
#define ROAD_LENGTH 19U
#define ROAD_MASK ((1U << ROAD_LENGTH) - 1U)
#define STOP_LINE_INDEX 7U
#define CAR_STEP_MS 800U
#define DISPLAY_REFRESH_MS 800U
#define FLOW_SAMPLE_MS 100U
#define LIGHT_YELLOW_MS     1500U
#define LIGHT_GREEN_MIN_MS  5000U
#define LIGHT_GREEN_MAX_MS 10000U
#define LIGHT_RED_MIN_MS    5000U
#define LIGHT_RED_MAX_MS   10000U
```

```c
int main(void)
{

    data_Rst();
    /* Initialize LEDs and ADC GPIOs */
    gpio_init();
    printf("GPIO init Done\n");

    /* set to 1 to run hardware check */
    if(0){
        printf("Hardware Check");

        GPIO_SetBits(LED_RED);
        GPIO_SetBits(LED_YEL);
        GPIO_SetBits(LED_GRN);

        for(int i =0; i<30 ; i++){
                data_High();
                delay(500); //delay 500ms
                data_Low();
                delay(500); //delay 500ms
            }
    }
    /* Create queues for inter-task communication */
    flow_to_traffic_q = xQueueCreate(1, sizeof(uint16_t));
    flow_to_light_q = xQueueCreate(1, sizeof(uint16_t));
    light_to_traffic_q = xQueueCreate(1, sizeof(uint8_t));
    light_to_display_q = xQueueCreate(1, sizeof(uint8_t));
    road_to_display_q = xQueueCreate(1, sizeof(uint32_t));
    light_event_q = xQueueCreate(1, sizeof(uint8_t));

    /* One-shot timer; callback posts a light-change event. */
    light_timer = xTimerCreate("LightTimer",
                               pdMS_TO_TICKS(LIGHT_RED_MAX_MS),
                               pdFALSE,
                               NULL,
                               light_timer_callback);

    /* Check if any of the queues or timer creation failed */
    if (light_timer == NULL || flow_to_traffic_q == NULL || flow_to_light_q == NULL ||
        light_to_traffic_q == NULL || light_to_display_q == NULL ||
        road_to_display_q == NULL || light_event_q == NULL) {
        for (;;);
    }

    /* Create tasks after creating queues and timers to ensure they are available when the tasks start running */
    xTaskCreate( flow_adjust_Task, "Flow_Adjust", configMINIMAL_STACK_SIZE, NULL, 2, NULL);
    xTaskCreate( traffic_gen_Task, "Traffic_Gen", configMINIMAL_STACK_SIZE, NULL, 1, NULL);
    xTaskCreate( light_controller_Task, "Light_Controller", configMINIMAL_STACK_SIZE, NULL, 2, NULL);
    xTaskCreate( display_Task, "Display", configMINIMAL_STACK_SIZE, NULL, 1, NULL);


    /* Start the tasks and timer running. */
    vTaskStartScheduler();

    return 0;
}
```

```c
/*---------------------Helper Functions--------------------*/

// Initialize hardware
static void gpio_init()
{
    RCC_AHB1PeriphClockCmd(RCC_AHB1Periph_GPIOC, ENABLE);   //enable clock for portC
    RCC_APB2PeriphClockCmd(RCC_APB2Periph_ADC1, ENABLE);    //enable clock for ADC1

    //LED's
    GPIO_InitTypeDef GPIO_Output_Config;
    GPIO_StructInit(&GPIO_Output_Config);                   //reset all members of the struct to default values
    GPIO_Output_Config.GPIO_Pin = (YEL | GRN | RED | CLR_PIN | CLK_PIN | DATA_PIN);     //we want to control the LEDs and shift register, so we need to set the corresponding pins
    GPIO_Output_Config.GPIO_Mode = GPIO_Mode_OUT;       //we want to control the LEDs, so we need output mode
    GPIO_Output_Config.GPIO_PuPd = GPIO_PuPd_NOPULL;    //no pull-up or pull-down resistors
    GPIO_Output_Config.GPIO_Speed = GPIO_Speed_2MHz;    //the speed of the output signal is not important, so we set it to the lowest
    GPIO_Output_Config.GPIO_OType = GPIO_OType_PP;      //push/pull
    GPIO_Init(GPIOC, &GPIO_Output_Config);

    //ADC
    GPIO_InitTypeDef GPIO_ADC_Config;
    GPIO_StructInit(&GPIO_ADC_Config);              //reset all members of the struct to default values
    GPIO_ADC_Config.GPIO_Pin = GPIO_Pin_3;          //we want to read the value of the potentiometer, which is connected to pin 3 of port C
    GPIO_ADC_Config.GPIO_Mode = GPIO_Mode_AN;       //we want to read the value of the potentiometer, so we need analog mode
    GPIO_ADC_Config.GPIO_PuPd = GPIO_PuPd_NOPULL;   //no pull-up or pull-down resistors
    GPIO_ADC_Config.GPIO_Speed = GPIO_Speed_2MHz;   //the speed of the output signal is not important, so we set it to the lowest
    GPIO_Init(GPIOC, &GPIO_ADC_Config);
    ADC_CommonInitTypeDef ADC_Common;       //we need to configure the common ADC settings, such as the clock and the mode of operation
    ADC_CommonStructInit(&ADC_Common);      //reset all members of the struct to default values
    ADC_Common.ADC_Mode = ADC_Mode_Independent; //use ADC1 only
    ADC_Common.ADC_Prescaler = ADC_Prescaler_Div4; //reduce ADC clock
    ADC_Common.ADC_DMAAccessMode = ADC_DMAAccessMode_Disabled; //no DMA
    ADC_Common.ADC_TwoSamplingDelay = ADC_TwoSamplingDelay_5Cycles; //default delay
    ADC_CommonInit(&ADC_Common);


    ADC_InitTypeDef ADC_Config;
    ADC_StructInit(&ADC_Config);                        //reset all members of the struct to default
    ADC_Config.ADC_ContinuousConvMode = DISABLE; //single conversions on demand
    ADC_Config.ADC_ExternalTrigConvEdge = ADC_ExternalTrigConvEdge_None; //software trigger only
    ADC_Config.ADC_DataAlign = ADC_DataAlign_Right;         //we want the data to be right-aligned, so we set the data alignment to right
    ADC_Init(ADC1,&ADC_Config);
    ADC_Cmd(ADC1, ENABLE);                      //enable the ADC
    ADC_RegularChannelConfig(ADC1, ADC_Channel_13, 1, ADC_SampleTime_3Cycles);  //we want to read the value of the potentiometer, channel 13 of the ADC, so we need to configure the regular channel
}

// Shift Register Data High
void data_High(){
    GPIO_ResetBits(SHIFT_PORT, CLK_PIN);
    GPIO_SetBits(SHIFT_PORT, CLR_PIN);
    GPIO_SetBits(SHIFT_PORT, DATA_PIN);
    // busy-wait ~1us instead of 10ms vTaskDelay
    volatile uint32_t c = (SystemCoreClock / 4000000U);
    while(c--) { __NOP(); }
    GPIO_SetBits(SHIFT_PORT, CLK_PIN);
    c = (SystemCoreClock / 4000000U);
    while(c--) { __NOP(); }
    GPIO_ResetBits(SHIFT_PORT, CLK_PIN);
    c = (SystemCoreClock / 4000000U);
    while(c--) { __NOP(); }
}
```

```c
void data_Low(){
    GPIO_ResetBits(SHIFT_PORT, CLK_PIN);
    GPIO_SetBits(SHIFT_PORT, CLR_PIN);
    GPIO_ResetBits(SHIFT_PORT, DATA_PIN);
    volatile uint32_t c = (SystemCoreClock / 4000000U);
    while(c--) { __NOP(); }
    GPIO_SetBits(SHIFT_PORT, CLK_PIN);
    c = (SystemCoreClock / 4000000U);
    while(c--) { __NOP(); }
    GPIO_ResetBits(SHIFT_PORT, CLK_PIN);
    c = (SystemCoreClock / 4000000U);
    while(c--) { __NOP(); }
}
// Turn off all LEDs on the shift register
void data_Rst(){
    GPIO_ResetBits(SHIFT_PORT, CLR_PIN);    // drives the reset line low
    delay(10);      // delay 10ms
    GPIO_SetBits(SHIFT_PORT, CLR_PIN);      // set reset to high
}

/*  Writes car data to the shift register.
    The data is a 19-bit value, where each bit represents a car in the 3 lanes.
    The MSB represents the oldest car, and the LSB represents the newest car.
    A bit value of 1 means there is a car, and a bit value of 0 means there is no car */
void screen_Write(uint32_t data){
    //data_Rst(); // clear display
    char car_To_Send = 0x0;

    for(int i=18;i>=0;i--){ //MSB first, so we start from the highest bit and shift down to the lowest bit
        car_To_Send = 0x1 & (data>>i);
        if(car_To_Send == 0x1){
            data_High();
        }else{
            data_Low();
        }
    }
}

// Reads the ADC
static uint16_t read_pot_adc(void)
{
    /* Start a single ADC conversion and return the 12-bit value. */
    ADC_SoftwareStartConv(ADC1);
    while (ADC_GetFlagStatus(ADC1, ADC_FLAG_EOC) == RESET) {    //wait for conversion to complete
    }
    return (uint16_t)ADC_GetConversionValue(ADC1);
}
```

```c
static uint32_t road_step(uint32_t road_bits, uint8_t light_state)
{
    /* Shift cars forward by one LED per tick. */
    uint32_t moved = 0U;

    if (light_state == LIGHT_GRN) {
        moved = (road_bits << 1) & ROAD_MASK; //advance every car one position, drop anything past the road length
        return moved;
    }

    /* When yellow or red, cars must queue at the stop line. */
    uint32_t lower_mask = (1U << (STOP_LINE_INDEX + 1U)) - 1U; //bits from 0..stop line
    uint32_t upper_mask = ROAD_MASK & ~lower_mask; //bits past the stop line
    uint32_t lower = road_bits & lower_mask;
    uint32_t upper = (road_bits & upper_mask) << 1; //cars already past the line keep moving

    /* Move cars forward only if the next position is free (single-lane queue). */
    for (int i = (int)STOP_LINE_INDEX - 1; i >= 0; i--) {
        uint32_t cur_bit  = 1U << (uint32_t)i;
        uint32_t next_bit = 1U << (uint32_t)(i + 1);
        if ((lower & cur_bit) != 0U && (lower & next_bit) == 0U) {
            lower &= ~cur_bit;
            lower |= next_bit;
        }
    }

    moved = (upper | lower) & ROAD_MASK;
    return moved;
}

// RNG algorithm
static uint32_t rng_next(uint32_t *state)
{
    /* Simple xor shift RNG for car spawning decisions. */
    uint32_t x = *state;    // read the current state
    x ^= x << 13;           // xor shift operations to produce the next state
    x ^= x >> 17;           // these operations are designed to produce a good distribution of random values while being computationally efficient
    x ^= x << 5;            // the specific shift values (13, 17, 5) are commonly used in xor shift RNGs and have been found to work well empirically
    *state = x;             // write the new state back to the variable pointed to by state
    return x;
}

// Called when the light timer expires, and it posts an event to the light controller task to indicate that it's time to change the light state.
static void light_timer_callback(TimerHandle_t timer)
{
    /* Post an event to advance the light state. */
    uint8_t event = 1;
    (void)timer;
    // we don't actually need to send any data with the event, since the light controller just needs to know that the timer has expired
    // and can use its existing state to determine the next light state, but we need to send something since the queue doesn't support sending empty messages
    if (light_event_q != NULL) {
        xQueueOverwrite(light_event_q, &event);
    }
}
```

```c
void delay(uint32_t ms)
{
    if (ms == 0U) {
        return;
    }

    if (xTaskGetSchedulerState() != taskSCHEDULER_NOT_STARTED) {
        /* Yield to other tasks when the scheduler is running. */
        TickType_t ticks = pdMS_TO_TICKS(ms);
        if (ticks > 0U) {
            vTaskDelay(ticks);
            return;
        }
    }

    /* Fallback for early init or sub-tick delays: simple CPU busy-wait. */
    volatile uint32_t count = (SystemCoreClock / 8000U) * ms;
    while (count-- != 0U) {
        __NOP();
    }
}

/*------------------------Tasks-----------------------------*/

/* Reads pot value and adjusts traffic flow based on it */
static void flow_adjust_Task( void *pvParameters )
{
    TickType_t last_wake = xTaskGetTickCount(); //initialize the last wake time variable for vTaskDelayUntil
    uint16_t flow_value = 0;

    (void)pvParameters; //avoid unused parameter warning, we don't need to pass any parameters to this task

    for (;;) {  // Run forever until task is stopped
        flow_value = read_pot_adc();
        printf("Flow Adjustment Task Running -- ");
        printf("Flow Value: %u\n", flow_value);

        if (flow_to_traffic_q != NULL) { //send to traffic generator
            xQueueOverwrite(flow_to_traffic_q, &flow_value);
        }
        if (flow_to_light_q != NULL) { //send to light controller
            xQueueOverwrite(flow_to_light_q, &flow_value);
        }

        vTaskDelayUntil(&last_wake, pdMS_TO_TICKS(FLOW_SAMPLE_MS));
    }
}
```

ewg > Downloads > C main.c

```c
/* random traffic generator */
static void traffic_gen_Task( void *pvParameters )
{
    TickType_t last_wake = xTaskGetTickCount();
    uint16_t flow_value = 0;
    uint32_t car_bits = 0;  // bitfield representing the presence of cars on the road, where each bit is a position on the road and a value of 1 means there is a car at that position
    uint32_t rng_state = 0x1234ABCD;    // initial state for the random number generator, can be any non-zero value
    uint8_t light_state = LIGHT_RED;
    uint8_t spawn_countdown = 1;

    (void)pvParameters;

    for (;;) {
        uint16_t flow_update = 0;
        uint8_t light_update = 0;
        // check for updates from flow adjuster and light controller, but don't block if they haven't sent anything
        if (flow_to_traffic_q != NULL && xQueueReceive(flow_to_traffic_q, &flow_update, 0) == pdPASS) {
            flow_value = flow_update;
        }
        // if we haven't received an update from the light controller, we can just use the last known state, so we don't block waiting for an update
        if (light_to_traffic_q != NULL && xQueueReceive(light_to_traffic_q, &light_update, 0) == pdPASS) {
            light_state = light_update;
        }

        car_bits = road_step(car_bits, light_state); //move cars one step

        if (spawn_countdown > 0) {
            spawn_countdown--;
        }

        if (spawn_countdown == 0) {
            uint32_t roll = rng_next(&rng_state) & FLOW_MAX_VALUE;
            if (flow_value >= (FLOW_MAX_VALUE - 1U) || roll < flow_value) {
                if ((car_bits & 0x1U) == 0U) {
                    car_bits |= 0x1U;

                    printf("Car spawned! Road: 0x%05lX\n", (unsigned long)car_bits);
                }
            }

            /* Guard: if flow exceeds max, clamp it before subtraction */
            uint16_t clamped_flow = (flow_value > FLOW_MAX_VALUE) ? FLOW_MAX_VALUE : flow_value;
            spawn_countdown = (uint8_t)(1U + ((FLOW_MAX_VALUE - clamped_flow) * 5U) / FLOW_MAX_VALUE);
            if (spawn_countdown == 0U) {
                spawn_countdown = 1U;
            }
        }

        if (road_to_display_q != NULL) {
            xQueueOverwrite(road_to_display_q, &car_bits);
        }

        vTaskDelayUntil(&last_wake, pdMS_TO_TICKS(CAR_STEP_MS));
    }
}
```

Do you want to install the re
Extension Pack' extension f
language?

```c
static void light_controller_Task( void *pvParameters )
{
    uint16_t flow_value = 0;
    uint8_t light_state = LIGHT_RED;    //initial light state is red
    uint8_t timer_event = 0;

    (void)pvParameters; // avoid unused parameter warning, we don't need to pass any parameters to this task
    // Initialize display and traffic generator with initial light state
    if (light_to_display_q != NULL) {
        xQueueOverwrite(light_to_display_q, &light_state);
    }
    // Traffic generator needs to know the initial light state to properly hold cars at the stop line, so we send it the initial state as well
    if (light_to_traffic_q != NULL) {
        xQueueOverwrite(light_to_traffic_q, &light_state);
    }
    // Start the light timer with the initial red duration
    if (light_timer != NULL) { //start with red interval
        xTimerChangePeriod(light_timer, pdMS_TO_TICKS(LIGHT_RED_MAX_MS), 0);
        xTimerStart(light_timer, 0);
    }

    for (;;) {
        uint16_t flow_update = 0;
        if (flow_to_light_q != NULL && xQueueReceive(flow_to_light_q, &flow_update, 0) == pdPASS) {
            flow_value = flow_update;
        }
        // wait for a timer event to change the light state, but don't block waiting for flow updates
        if (light_event_q != NULL && xQueueReceive(light_event_q, &timer_event, portMAX_DELAY) == pdPASS) {
            /* Map flow to green/red durations. */
            uint32_t green_ms = LIGHT_GREEN_MIN_MS +
                                ((uint32_t)flow_value * (LIGHT_GREEN_MAX_MS - LIGHT_GREEN_MIN_MS)) / FLOW_MAX_VALUE;
            uint32_t red_ms = LIGHT_RED_MAX_MS -
                                ((uint32_t)flow_value * (LIGHT_RED_MAX_MS - LIGHT_RED_MIN_MS)) / FLOW_MAX_VALUE;

            if (light_state == LIGHT_RED) { //red -> green
                light_state = LIGHT_GRN;
                if (light_timer != NULL) {
                    xTimerChangePeriod(light_timer, pdMS_TO_TICKS(green_ms), 0);
                    xTimerStart(light_timer, 0);
                }
            } else if (light_state == LIGHT_GRN) { //green -> yellow
                light_state = LIGHT_YEL;
                if (light_timer != NULL) {
                    xTimerChangePeriod(light_timer, pdMS_TO_TICKS(LIGHT_YELLOW_MS), 0);
                    xTimerStart(light_timer, 0);
                }
            } else { //yellow -> red
                light_state = LIGHT_RED;
                if (light_timer != NULL) {
                    xTimerChangePeriod(light_timer, pdMS_TO_TICKS(red_ms), 0);
                    xTimerStart(light_timer, 0);
                }
            }

            uint8_t stale_event;
            while (xQueueReceive(light_event_q, &stale_event, 0) == pdPASS) {}

            // Update display and traffic generator with new light state
            if (light_to_display_q != NULL) {
                xQueueOverwrite(light_to_display_q, &light_state);
            }
            // Traffic generator needs to know the light state to properly hold cars at the stop line, so we send it the updated state as well
            if (light_to_traffic_q != NULL) {
                xQueueOverwrite(light_to_traffic_q, &light_state);
            }
        }
```

```c
/* Outputs to LEDs */
static void display_Task( void *pvParameters )
{
    TickType_t last_wake = xTaskGetTickCount();
    uint32_t road_bits = 0;
    uint8_t light_state = LIGHT_RED;

    (void)pvParameters; // avoid unused parameter warning, we don't need to pass any parameters to this task

    for (;;) {
        uint32_t road_update = 0;
        uint8_t light_update = 0;
        // check for updates from traffic generator and light controller, but don't block if they haven't sent anything,
        //since we want to refresh the display at a consistent rate regardless of whether we've received updates
        if (road_to_display_q != NULL && xQueueReceive(road_to_display_q, &road_update, 0) == pdPASS) {
            road_bits = road_update;
        }
        // if we haven't received an update from the light controller, we can just use the last known state, so we don't block waiting for an update
        if (light_to_display_q != NULL && xQueueReceive(light_to_display_q, &light_update, 0) == pdPASS) {
            light_state = light_update;
        }

        GPIO_ResetBits(LED_RED);
        GPIO_ResetBits(LED_YEL);
        GPIO_ResetBits(LED_GRN);

        if (light_state == LIGHT_GRN) { //set traffic light LEDs
            GPIO_SetBits(LED_GRN);
        } else if (light_state == LIGHT_YEL) {
            GPIO_SetBits(LED_YEL);
        } else {
            GPIO_SetBits(LED_RED);
        }

        screen_Write(road_bits & ROAD_MASK); //push road state to shift register

        vTaskDelayUntil(&last_wake, pdMS_TO_TICKS(DISPLAY_REFRESH_MS)); //refresh display at a consistent rate
    }
}
```